CORPORACIÓN UNIVERSITARIA DEL HUILA
CORHUILA

ATTENDANCE REGISTRY

Informe final de métricas de mantenibilidad y evolución

Asignatura: Métricas y Estándares de Calidad

José Miguel Aya Peralta
Luis Fernando Pérez Chala
Andrés Felipe Calle González

Programa académico: Ingeniería en Sistemas
Docente: José Miguel Llanos Mosquera
Neiva, Huila, Colombia
Período 2026-B · 9 de noviembre de 2026

Índice

1. Identificación del proyecto **2**

2. Introducción **2**

3. Requisitos del sistema **2**
3.1. Requisitos funcionales . . . 2
3.2. Requisitos no funcionales . . . 3

4. Arquitectura del sistema **3**

5. Implementación y evolución **4**

6. Definición y procedimiento de métricas **4**
6.1. Complejidad ciclomática V(G) . . . 4
6.2. Líneas de código (LOC) . . . 4
6.3. Suite CK . . . 4
6.4. Reproducibilidad y comparabilidad . . . 4

7. Resultados y evidencias **5**
7.1. PMD: antes y después . . . 5
7.2. LOC y documentación interna . . . 5
7.3. Métricas CK . . . 5
7.4. Pruebas automatizadas y funcionales . . . 6
7.5. Seguridad y rendimiento: alcance . . . 6

8. Interpretación y plan de mejoras **6**

9. Conclusiones **7**

10. Aportes individuales y colaboración **7**

11. Referencias y fuentes de evidencia **8**
11.1. Referencias académicas y de herramientas . . . 8
11.2. Evidencia primaria del repositorio . . . 8

1. Identificación del proyecto

Nombre del sistema: Attendance Registry (Att-book). **Naturaleza:** aplicación web para administrar cursos, sesiones académicas y asistencia de estudiantes mediante códigos QR temporales. **Propósito de esta entrega:** evaluar la mantenibilidad de una aplicación real, documentar las métricas V(G), LOC y CK, aplicar una refactorización fundamentada y comparar las mediciones antes y después.

El trabajo utiliza una versión integrada del proyecto de Fullstack y distingue la procedencia del código original de las actividades académicas de integración, medición, documentación y refactorización. El repositorio contiene componentes de procedencia privada, cuya autorización de redistribución debe verificarse con los titulares antes de cerrar el proceso de publicación.

2. Introducción

El control manual de asistencias puede producir registros duplicados, errores de consolidación y dificultades para consultar historiales. Attendance Registry permite a docentes administrar cursos y sesiones, emitir códigos QR de vigencia limitada y revisar registros; los estudiantes pueden registrarse, autenticarse e informar su asistencia mediante un código válido.

El interés de la asignatura no se limita a demostrar que el sistema funciona: también busca valorar qué tan comprensible y modificable resulta su código. Para ello se usan tres grupos de métricas complementarios: complejidad ciclomática por método, líneas de código y métricas de diseño orientado a objetos. Los resultados sirven para identificar riesgos de mantenimiento, justificar modificaciones concretas y formular nuevas acciones de mejora.

Objetivo general. Evaluar la mantenibilidad del backend de Attendance Registry mediante mediciones estáticas, pruebas y una refactorización verificable.

Objetivos específicos. (i) Documentar los requisitos y la arquitectura; (ii) medir complejidad, tamaño y propiedades de diseño del backend Java; (iii) identificar métodos con complejidad superior a la meta establecida; (iv) refactorizar conservando el comportamiento esperado; (v) repetir mediciones comparables y analizar efectos favorables y desfavorables; (vi) priorizar mejoras futuras.

Alcance. La medición principal se concentra en el backend Java de producción. El portal React/TypeScript se valida mediante pruebas, lint y compilación, pero no forma parte de los CSV de CK ni del conteo CLOC del backend. Los resultados de rendimiento y seguridad corresponden únicamente a los escenarios registrados y al entorno local de prueba.

3. Requisitos del sistema

3.1. Requisitos funcionales

Los diez requisitos funcionales definidos en `docs/requisitos/requisitos.md` aparecen implementados y se registraron como aprobados en la verificación manual de integración.

Código	Requisito	Actor / alcance	Resultado documentado
RF-01	Registro de estudiantes	Estudiante	PASS
RF-02	Autenticación y control de acceso	Docente / estudiante	PASS
RF-03	Creación de cursos	Docente	PASS
RF-04	Consulta de cursos propios	Docente	PASS
RF-05	Actualización de cursos	Docente	PASS
RF-06	Creación de sesiones académicas	Docente	PASS
RF-07	Consulta de sesiones	Usuarios autorizados	PASS
RF-08	Generación de QR temporales	Docente	PASS
RF-09	Registro de asistencia mediante QR	Estudiante inscrito	PASS
RF-10	Consulta de asistencias por sesión	Docente autorizado	PASS

Entre los criterios relevantes se encuentran impedir el registro con QR vencidos o inactivos, rechazar estudiantes no inscritos, evitar asistencias duplicadas y restringir operaciones sobre recursos ajenos. La matriz de trazabilidad enlaza cada requisito con componentes concretos y evidencias de prueba. La aprobación manual de cada RF no implica que todos sus posibles casos límite tengan una prueba automatizada individual.

3.2. Requisitos no funcionales

Código	Criterio de aceptación	Resultado registrado	Alcance de la conclusión
RNF-01	V(G) ≤ 10 por método del backend analizado	PMD: 3 infracciones antes, 0 después	Cumple según el ruleset PMD aplicado
RNF-02	Rechazar solicitudes sin autenticación o permisos	40/40 escenarios PASS	Escenarios de estados HTTP previstos; no auditoría integral
RNF-03	P95 ≤ 2.000 ms bajo carga definida	P95 global 110,85 ms	Cuatro endpoints GET, Docker local, 20 VU

La trazabilidad de RF y RNF se encuentra en `docs/requisitos/requisitos.md`, y los resultados en `docs/pruebas/` y `docs/metricas/`.

4. Arquitectura del sistema

Attendance Registry presenta una arquitectura **cliente-servidor** con un **backend monolítico modular organizado en capas**. El portal utiliza React y TypeScript; la API implementa controladores, servicios de negocio y repositorios mediante Java 17 y Spring Boot. Spring Security y JWT permiten autenticar y autorizar solicitudes; PostgreSQL 17 conserva las entidades; Docker Compose y Nginx facilitan el despliegue local.

Flujo principal: usuario (docente o estudiante) → portal web → API REST protegida → controladores → servicios de negocio → repositorios JPA → PostgreSQL. La respuesta vuelve por el mismo recorrido hacia el portal.

Módulos: identidad y autenticación; cursos; inscripciones; sesiones; generación y validación de códigos QR; registro y consulta de asistencias. Las clases representativas incluyen `AuthController`, `CourseService`, `SessionService`, `QrService`, `AttendanceService` y sus repositorios. El proyecto integra cinco componentes de origen en `sistema/`: API, portal, base de datos, infraestructura y documentación.

Justificación tecnológica. Java y Spring Boot mantienen encapsuladas las reglas de negocio y facilitan aplicar PMD, CK, JUnit y Mockito. React y TypeScript permiten desarrollar la interfaz con componentes verificados mediante Vitest. PostgreSQL soporta las relaciones y restricciones de integridad; Docker Compose reproduce los servicios necesarios en un entorno local. Esta elección reutiliza el stack de Fullstack, evitando una reescritura ajena al objetivo de la asignatura.

Ventajas: separación básica entre interfaz, reglas de negocio y persistencia; facilidad de pruebas por módulo; despliegue integrado. **Compromisos:** mayor concentración de responsabilidades en algunos servicios, controladores con dependencias elevadas y coordinación necesaria entre cambios de API y portal. Los indicadores CK presentados más adelante respaldan estos riesgos.

Representación arquitectónica resumida:

```
[Docentes / estudiantes]
          |
[Portal React + TypeScript / Nginx]
          |
[API REST - Spring Security + JWT]
          |
[Controladores: identidad, cursos, sesiones, QR, asistencia]
          |
[Servicios de negocio / repositorios JPA]
          |
[PostgreSQL 17]
```

La versión del diagrama Mermaid y la descripción completa de componentes se conservan en `docs/arquitectura/arquitectura.md`.

5. Implementación y evolución

La aplicación permite registrar y autenticar estudiantes, gestionar cursos y sesiones académicas, emitir códigos QR temporales y consultar asistencias de acuerdo con las reglas de acceso. En los registros de integración se documentaron diez pruebas funcionales manuales exitosas y un despliegue local con PostgreSQL, API y portal en ejecución.

La refactorización de esta entrega se centró en tres métodos del backend que superaban el umbral de V(G) requerido. En el PR #6 se utilizó principalmente la técnica **Extract Method**, separando validaciones y comprobaciones de autorización sin redefinir el contrato funcional de la aplicación:

Clase	Método original	V(G) previo
`SessionService`	`validateRequest`	17
`AttendanceService`	`registerAttendance`	15
`SessionService`	`requireCourseAccess`	11

Se incorporaron pruebas específicas para `AttendanceService` y se conservaron los resultados de PMD anteriores y posteriores. La ausencia de infracciones en el reporte posterior acredita el cumplimiento del umbral configurado; no equivale, por sí sola, a demostrar que todas las métricas de diseño mejoraron.

6. Definición y procedimiento de métricas

6.1. Complejidad ciclomática V(G)

La complejidad ciclomática estima los caminos linealmente independientes del flujo de control. Su expresión general es **V(G) = E − N + 2P**, donde E son aristas, N nodos y P componentes conectados; para un método con una sola componente suele expresarse en términos de decisiones como **D + 1**, de acuerdo con las convenciones de conteo de la herramienta.

Herramienta: PMD 7.17.0, integrado con `maven-pmd-plugin` 3.28.0 y reglas en `sistema/att-book-api/config/pmd-complejidad.xm`
Criterio de aceptación del proyecto: V(G) ≤ 10; las infracciones se registran cuando el umbral se supera. **Evidencias:** reportes XML en `docs/metricas/pmd-final/` y `docs/metricas/pmd-post-refactor/`.

6.2. Líneas de código (LOC)

LOC cuantifica el tamaño del código. Se distingue **líneas físicas** (total), **líneas de código según CLOC**, **comentarios** y **líneas en blanco**. Para la versión actual, el conteo se hizo con **CLOC 2.08** sobre `sistema/att-book-api/src/main/java`, excluyendo las pruebas.

Para las longitudes de métodos se utilizó la columna `loc` de `docs/metricas/ck-post-refactor/method.csv`. El umbral académico es **más de 50 LOC por método** como señal de alerta. Los dos instrumentos tienen convenciones distintas y no deben presentar sus métricas como intercambiables. El porcentaje de comentarios se calculó como comentarios / líneas físicas × 100.

6.3. Suite CK

CK (Chidamber y Kemerer) caracteriza clases orientadas a objetos mediante WMC (complejidad de métodos por clase), DIT (profundidad de herencia), NOC (cantidad de descendientes directos), CBO (acoplamiento), RFC (respuesta potencial de la clase) y LCOM (falta de cohesión). En los archivos CSV se distingue `lcom` de `lcom*`, la variante normalizada.

Herramienta: CK de Mauricio Aniche, versión exacta no identificada en los archivos recibidos. **Umbrales de alerta adoptados:** WMC > 20, DIT > 5, CBO > 10, RFC > 50 y LCOM* > 0,7. No se fija umbral numérico general de NOC. Estos umbrales son indicadores orientativos, no pruebas automáticas de defectos.

6.4. Reproducibilidad y comparabilidad

Los reportes separan tres etapas: **base inicial**, **versión funcional integrada anterior a la refactorización** y **versión posterior a la refactorización**. Las comparaciones de métodos y CK deben corresponder a etapas equivalentes; la comparación de LOC de la base inicial con la versión ampliada refleja también crecimiento funcional y, por ello, no mide exclusivamente el efecto de refactorizar.

La medición actual CLOC puede reproducirse con `cloc sistema/att-book-api/src/main/java --include-lang=Java --csv`, usando CLOC 2.08. La ejecución de las pruebas de arranque requiere PostgreSQL de pruebas y variables `DB_URL`, `DB_USERNAME`, `DB_PASSWORD` y `JWT_SECRET`, sin reutilizar credenciales de producción.

7. Resultados y evidencias

7.1. PMD: antes y después

Resultado	Antes	Después
Métodos por encima del umbral de V(G)	3	0

Los métodos iniciales con valores 17, 15 y 11 dejaron de aparecer como infracciones después de aplicar Extract Method. Esto respalda el cumplimiento del RNF-01 **en los métodos y configuración analizados**.

7.2. LOC y documentación interna

Indicador CLOC (backend actual)	Total
Archivos Java de producción	43
Líneas de código	2.017
Líneas de comentarios	31
Líneas en blanco	489
Líneas físicas	2.537
Comentarios / líneas físicas	1,22 %

La medida anterior de versión ampliada registró 2.473 líneas físicas y 1.994 no vacías, pero usó otra clasificación. Se conserva como referencia histórica, **no** como una comparación exacta de código efectivo con CLOC.

El archivo CK posterior contiene **336 registros de métodos y constructores**, de los cuales **305 son métodos no constructores**. Al aplicar el filtro `constructor != true` y `loc > 50`, **no se hallaron métodos por encima de 50 LOC**. La mayor LOC registrada en el CSV es 19, incluidos los constructores. Esta conclusión corresponde a la definición de `loc` usada por CK; no acredita por sí sola todas las posibles reglas alternativas de conteo de líneas.

7.3. Métricas CK

Las medias de producción que constan en el informe comparativo son:

Métrica	Antes de refactorizar	Después
WMC	4,79	5,04
DIT	1,02	1,02
NOC	0,00	0,00
CBO	5,60	5,60
RFC	5,62	5,88
LCOM (bruto)	2,56	4,94

El informe comparativo describe una selección de **48 clases e interfaces de producción** para estas medias. Los CSV originales pueden incluir otros registros (por ejemplo, tipos anidados), de modo que esas medias deben interpretarse con el filtro de producción utilizado y no como el promedio directo de todas las filas brutas del CSV.

En dos servicios los resultados puntuales fueron:

Clase	WMC antes → después	LCOM* antes → después
`SessionService`	38 → 44	0,694 → 0,796
`AttendanceService`	18 → 24	0,520 → 0,764

También se registran **CBO = 19** en `CourseController` y `SessionController`, por encima de la referencia de 10. El aumento de WMC y LCOM* en los servicios no invalida la reducción de V(G) por método; revela otras dimensiones de mantenibilidad que aún exigen revisión.

7.4. Pruebas automatizadas y funcionales

Verificación documentada	Resultado
Pruebas automatizadas backend, JUnit/Mockito/Maven	134/134 aprobadas
Pruebas automatizadas frontend, Vitest	27/27 aprobadas
Pruebas funcionales manuales RF-01 a RF-10	10/10 PASS
Seguridad RNF-02	40/40 escenarios PASS
Rendimiento RNF-03	400/400 comprobaciones PASS
Revisión estática frontend	0 errores y 1 advertencia de lint
Compilación frontend	Exitosa; advertencia de paquete > 500 kB

El registro de Maven indica compilación y ejecución exitosas con PostgreSQL 17 de pruebas; la prueba `AttBookApiApplicationTests.cont` requirió configurar el entorno antes de pasar. Estos son resultados **documentados por el equipo**, no pruebas ejecutadas durante la preparación de este PDF.

7.5. Seguridad y rendimiento: alcance

Para RNF-02 se verificaron 40 respuestas HTTP en escenarios de petición sin autenticación, token inválido y permisos de rol incorrectos; todos aprobaron. Esto **no** representa una auditoría integral de vulnerabilidades, permisos por recurso o dependencias.

Para RNF-03, k6 utilizó 20 usuarios virtuales, cinco iteraciones por usuario y cuatro endpoints de consulta solicitados secuencialmente en cada iteración. Se completaron 400 solicitudes GET de prueba más dos solicitudes de autenticación, sin fallos HTTP, y el P95 global fue 110,85 ms frente al límite de 2.000 ms.

Endpoint GET	Solicitudes	P95 registrado
Cursos del docente	100	106,83 ms
Historial de asistencias	100	57,83 ms
Inscripciones del estudiante	100	83,30 ms
Perfil del estudiante	100	116,39 ms

No se ejecutaron escenarios de escritura, y la carga se aplicó en Docker local, no en producción.

8. Interpretación y plan de mejoras

Hallazgo principal. La refactorización resolvió las tres infracciones de complejidad ciclomática; sin embargo, empeoraron algunos indicadores agregados de clase. Esto evidencia un compromiso: dividir métodos puede simplificar su flujo de control, pero incrementar el número de métodos y revelar responsabilidades poco cohesivas.

Prioridad	Evidencia	Acción propuesta	Criterio de verificación futuro
Alta	`SessionService`: WMC 44; LCOM* 0,796	Evaluar Extract Class y redistribuir validaciones, acceso y operaciones	Menor WMC/LCOM* sin regresiones

Prioridad	Evidencia	Acción propuesta	Criterio de verificación futuro
Alta	`AttendanceService`: WMC 24; LCOM* 0,764	Separar autorización, validación QR y persistencia cuando corresponda	Cohesión mejorada con pruebas de asistencia verdes
Media	`CourseController` y `SessionController`: CBO 19	Revisar dependencias e interfaces entre capas	CBO menor o dependencias justificadas
Media	31 líneas de comentarios de 2.537 físicas	Documentar reglas de negocio difíciles y contratos	Código y documentación revisados por el equipo
Media	k6 solo midió GET	Crear escenarios controlados de creación y asistencia	P95 y tasas de error en escrituras
Media	Advertencias React y tamaño de bundle	Revisar estado en efectos y carga diferida	Menos advertencias y tamaño optimizado

Priorización. Las dos primeras acciones se consideran altas por afectar los servicios directamente modificados. El resto se plantea como evolución futura; ninguna de estas propuestas se reporta como implementada en esta entrega. Cada acción debe acompañarse de pruebas de regresión y nuevas mediciones para comprobar su efecto real.

Limitaciones del estudio. El análisis se concentra en Java; la versión exacta de CK no está identificada; algunos conjuntos históricos usan metodologías distintas; las pruebas no cubren todos los riesgos de seguridad ni el rendimiento de escritura; no hay datos de un despliegue de producción. Las métricas señalan aspectos que merecen revisión, no determinan automáticamente la calidad global ni la ausencia de defectos.

9. Conclusiones

1. Se aplicaron métricas complementarias de complejidad, tamaño y diseño orientado a objetos sobre una aplicación real integrada, con reportes reproducibles conservados en el repositorio.
2. La refactorización mediante Extract Method redujo las infracciones PMD de **3 a 0**, cumpliendo RNF-01 según el umbral y configuración utilizados.
3. La medición CLOC del backend actual identificó **2.017 líneas de código**, **31 de comentarios** y **489 en blanco**, dentro de **2.537 líneas físicas** repartidas en 43 archivos.
4. El conjunto CK posterior incluyó 336 registros, de ellos 305 métodos no constructores; no se detectaron métodos de más de 50 LOC según la métrica CK empleada.
5. El aumento de WMC y LCOM* en `SessionService` y `AttendanceService` demuestra que una reducción local de complejidad no garantiza mejoras globales de cohesión o responsabilidad de clase.
6. Las evidencias documentadas indican aprobación de 134 pruebas backend, 27 frontend, 10 manuales, 40 escenarios de seguridad y 400 comprobaciones de carga, con limitaciones explícitas para escenarios no cubiertos.
7. El próximo ciclo de mantenimiento debería priorizar servicios con WMC/LCOM* altos, controladores acoplados y ampliación de pruebas de autorización y rendimiento de escritura.

10. Aportes individuales y colaboración

Integrante	Aportes registrados en el repositorio académico
José Miguel Aya Peralta	Integración de componentes, requisitos y arquitectura, pruebas funcionales, seguridad y rendimiento, refactorización RNF-01 y pruebas de asistencia, cierre técnico de documentación
Luis Fernando Pérez Chala	Configuración de PMD, CK y CLOC, mediciones y comparaciones, análisis de mantenibilidad, plan de mejoras, documentación de los resultados y consolidación del presente informe
Andrés Felipe Calle González	Figura como integrante del grupo; la documentación recibida no identifica commits técnicos propios atribuibles a él en el repositorio académico

La tabla describe evidencias del **repositorio académico** y no atribuye a ninguno de los integrantes la autoría exclusiva del código original incorporado desde los repositorios de Fullstack. Si existen aportes fuera de GitHub, deben registrarse con evidencia comprobable antes de la entrega final.

La trazabilidad principal incluye PR #5 (integración), PR #6 (refactorización), PR #7 (métricas comparativas), PR #8 (validación backend) y PR #10 (LOC y cierre de métricas). Los identificadores originales de los cinco repositorios privados se encuentran en `docs/integracion/versiones-origen.md`.

11. Referencias y fuentes de evidencia

11.1. Referencias académicas y de herramientas

- Llanos Mosquera, J. M. *Proyecto Final: Métricas de Mantenibilidad y Evolución aplicadas a una Aplicación Real*. Guía de la asignatura Métricas y Estándares de Calidad (referencia consignada en la documentación del proyecto).
- McCabe, T. J. (1976). *A Complexity Measure*. IEEE Transactions on Software Engineering.
- Chidamber, S. R., y Kemerer, C. F. (1994). *A Metrics Suite for Object Oriented Design*. IEEE Transactions on Software Engineering.
- PMD. *PMD Source Code Analyzer*, versión 7.17.0.
- AlDanial. *CLOC – Count Lines of Code*, versión 2.08.
- Aniche, M. *CK – Java Code Metrics Calculator*. Versión por confirmar a partir del ejecutable utilizado.
- Grafana Labs. *k6*, herramienta de pruebas de carga.

11.2. Evidencia primaria del repositorio

- `README.md`: alcance, integrantes, ejecución y resultados resumidos.
- `docs/requisitos/requisitos.md`: RF, RNF y trazabilidad.
- `docs/arquitectura/arquitectura.md`: módulos, decisiones y arquitectura.
- `docs/metricas/definicion-metricas.md` y `docs/metricas/informe-comparativo-final.md`: definiciones e interpretación.
- `docs/metricas/pmd-final/pmd.xml` y `docs/metricas/pmd-post-refactor/pmd.xml`: complejidad.
- `docs/metricas/ck-final/`, `docs/metricas/ck-post-refactor/` y `docs/metricas/loc-final/`: métricas CK y CLOC.
- `docs/pruebas/integration-tests.md`, `docs/pruebas/seguridad/` y `docs/pruebas/rendimiento/`: evidencia funcional, de seguridad y de carga.
- `docs/integracion/versiones-origen.md`: procedencia técnica y consideraciones de autoría.